LOOPtLOOP Witness Integration Plan

Abracadoo participates. WitnessKey witnesses. LOOPtLOOP binds.

Plan version: v0.1 | Date: 2026-06-19 | Target substrate: Cloudflare Workers + D1

Filename: 20260619__LOOPtLOOP__PLAN__WITNESS-INTEGRATION__v0-1__cloudflare-mvi-roadmap.docx

1. Executive Summary

Minimum viable LOOPtLOOP is not a new standalone app. It is a witnessed, consent-bound co-presence event where Abracadoo acts as a participant/runtime, WitnessKey acts as the witness/certificate surface, and LOOPtLOOP supplies the event grammar, consent loop, receipt, and verification API.

The first build should be an External WitnessMark Loop: WitnessMark creates a content fingerprint, Abracadoo consents as participant, LOOPtLOOP signs the bounded witness event, and WitnessKey displays and verifies the receipt.

2. Source Basis

- LOOPtLOOP documents define a two-way TOTP consent loop, API concepts, field anchors, charms, NFC/ESP32 ritual hardware, and vertical applications across toys, healthcare, gaming, Web3, and sacred/geocache-style field infrastructure.
- WitnessMark index.html already contains a working local SHA-256 receipt interface, explicit consent checkboxes, LOOPtLOOP internal co-presence mode, and an external LOOPtLOOP API placeholder seam.
- Abracadoo code at github.com/bobrs/abracadoo-code appears to already contain the right ontology seeds: local vault, Acquaintances, Paths, Loops, LoopWitness records, and future Relationship gating.
- The user's stated intent connects WitnessKey.online to witnessed server/client co-viewed content and positions Abracadoo.app as a real PWA participant with possible modifications.

3. Core Thesis

- WitnessKey proves that a moment happened around content.
- Abracadoo lets a participant consent to and carry that moment.
- LOOPtLOOP defines the loop that makes the event valid, bounded, reversible, inspectable, and portable.

Compressed formulation: Abracadoo participates. WitnessKey witnesses. LOOPtLOOP binds.

4. Revised Architecture

Layer	Existing asset	Role
Participant / path layer	abracadoo.app	Human-facing PWA that can initiate, accept, carry, display, and later re-present loop participation.
Witness / certificate layer	witnesskey.online / WitnessMark	Creates witnessed records from content co-viewed by server and client; displays receipts and verification.
Loop / consent layer	api.looptloop.io	Defines consent loop, state machine, presence proof, expiry, payload rules, signatures, and witness event schema.

```
Abracadoo participant
   <-> consent / presence loop
LOOPtLOOP API
   <-> witnessed event record
WitnessKey certificate / co-view proof
```

5. Minimum Viable Instance: External WitnessMark Loop

1. User opens WitnessKey/WitnessMark and enters or views content.
2. WitnessMark computes a SHA-256 content fingerprint and canonical receipt-material fingerprint.
3. WitnessMark creates a LOOPtLOOP witness offer through the Cloudflare-hosted API.
4. User is routed to Abracadoo.app to review the offer and consent as participant.
5. Abracadoo accepts the loop offer and stores a local receipt reference.
6. LOOPtLOOP creates and signs a bounded witness event.
7. WitnessKey displays the human-readable receipt and verification URL.
8. Later, anyone with the receipt can verify the event signature and content hash.

6. Consent UX Copy

Abracadoo accept screen should say:

```
WitnessMark is asking you to participate in a time-bound witness loop.

Subject: SHA-256 content fingerprint.
Stored: content hash, timestamp, consent event, receipt signature.
Not stored: raw content, unrelated browsing, legal identity.

This does not create a Relationship.
```

WitnessMark receipt should preserve its current non-claims: the receipt does not prove ownership, authorship, truth, legality, patent priority, external identity, absolute time, or legally valid consent.

7. Canonical Event Grammar

Define these three portable envelope profiles first:

Profile	Purpose
LOOPTLOOP_WITNESS_OFFER_1	Portable offer issued by WitnessKey/WitnessMark and accepted by Abracadoo or another participant runtime.
LOOPTLOOP_WITNESS_EVENT_1	Signed event envelope proving a bounded consent loop around a content fingerprint.
WITNESSMARK_RECEIPT_1	Human-readable receipt derived from the signed witness event.

Minimum witness event fields:

```
{
  "schema": "LOOPTLOOP_WITNESS_EVENT_1",
  "event_id": "we_...",
  "loop_id": "loop_...",
  "event_type": "content_co_witnessed",
  "participants": [
    {"role": "client_participant", "app": "abracadoo.app", "participant_ref":
"local_or_pseudonymous_path_id"},
    {"role": "server_witness", "app": "witnesskey.online", "origin":
"https://witnesskey.online"}
  ],
  "subject": {"type": "content_hash", "hash_alg": "SHA-256", "hash": "..."},
  "consent": {
    "prompt_hash": "...",
    "accepted_at": "...",
    "expires_at": "...",
    "allowed_actions": ["witness-content", "issue-receipt"]
  },
  "claims": {
    "does_claim": ["A participant consented to witness this content fingerprint during
the validity window."],
    "does_not_claim": ["authorship", "ownership", "truth", "legal consent", "legal
identity", "absolute time"]
  },
  "signature": "..."
}
```

8. Cloudflare API v0

Deploy the first API surface at api.looptloop.io. For v0, Workers + D1 are sufficient; Durable Objects can be added for live offer state or exactly-once semantics.

```
POST /v0/witness-offers
GET  /v0/witness-offers/:id
POST /v0/witness-offers/:id/accept
POST /v0/witness-offers/:id/reject
GET  /v0/witness-events/:id
GET  /v0/witness-events/:id/verify
```

9. Minimal D1 Schema

```
CREATE TABLE witness_offers (
  id TEXT PRIMARY KEY,
  issuer_origin TEXT NOT NULL,
  issuer_name TEXT NOT NULL,
  subject_hash_alg TEXT NOT NULL,
  subject_hash TEXT NOT NULL,
  canonical_material_hash TEXT,
  permissions_json TEXT NOT NULL,
  consent_prompt TEXT NOT NULL,
  created_at TEXT NOT NULL,
  expires_at TEXT NOT NULL,
  status TEXT NOT NULL
);

CREATE TABLE witness_events (
  id TEXT PRIMARY KEY,
  offer_id TEXT NOT NULL,
  loop_id TEXT NOT NULL,
  participant_app TEXT,
  participant_ref TEXT,
  issuer_origin TEXT NOT NULL,
  subject_hash_alg TEXT NOT NULL,
  subject_hash TEXT NOT NULL,
  consent_event_hash TEXT NOT NULL,
  receipt_json TEXT NOT NULL,
  receipt_signature TEXT NOT NULL,
  created_at TEXT NOT NULL,
  expires_at TEXT,
  status TEXT NOT NULL
);
```

10. Implementation Tasks

Track	Task	Output
Spec	Define offer/event/receipt schemas, canonicalization, expiry, signature, and verification response.	Versioned protocol doc.
WitnessMark	Replace runExternalLoopPlaceholder with real offer creation, Abracadoo redirect, polling, and receipt rendering.	External LOOPtLOOP witness mode.
Abracadoo	Add accept-loop route that fetches offer, displays consent, accepts/rejects, and stores local receipt.	Participant acceptance flow.
LOOPtLOOP API	Deploy Workers + D1 endpoints for offers, events, signatures, and verification.	api.looptloop.io v0.
WitnessKey	Add verify page showing event ID, content hash, issuer, participant app, consent condition, expiry, claims, and signature result.	Public receipt verification surface.

11. Roadmap

Phase 0 - Spec the envelope: Lock LOOPTLOOP_WITNESS_OFFER_1, LOOPTLOOP_WITNESS_EVENT_1, and WITNESSMARK_RECEIPT_1.

Phase 1 - Replace WitnessMark placeholder: External mode creates a real LOOPtLOOP offer and receives a signed event.

Phase 2 - Add Abracadoo accept route: Abracadoo becomes the participant runtime for external loop offers.

Phase 3 - Cloudflare API: Deploy v0 endpoints using Workers + D1.

Phase 4 - Verification page: WitnessKey verifies event receipts and explains claims/non-claims.

Phase 5 - Generalize to anchors: Use the same envelope for QR/NFC anchors, Memory Totems, field caches, toys, and ESP32 stations.

Phase 6 - SDK/pilot layer: Add JS SDK, partner docs, toy/game/field pilot flows, and eventually ESP32/NFC reference designs.

12. Boundaries and Non-Goals

- Do not make WitnessKey create Abracadoo Relationships. A witnessed content loop is an event, not a social relationship.
- Do not store raw witnessed content by default. Store hashes, signatures, consent events, and verification metadata.
- Do not make LOOPtLOOP depend specifically on Abracadoo. Abracadoo is the first participant runtime, not the only one.
- Do not require ESP32, NFC, Web3, accounts, or child-directed features for the first integration.
- Do not overclaim legal timestamping, authorship, truth, ownership, or legal consent.
- Do not let LOOPtLOOP become tracking with ritual language. Every loop must have visible consent, visible duration, visible stored fields, and visible revocation/expiry semantics.

13. Resume Point

When work resumes, start here: draft the exact LOOPTLOOP_WITNESS_OFFER_1 and LOOPTLOOP_WITNESS_EVENT_1 TypeScript interfaces, then patch the WitnessMark external placeholder to call a local/mock LOOPtLOOP API before deploying the Cloudflare Worker.

Appendix A. Source Inventory

Source	Relevance
LOOPtLOOP Field Infrastructure Applications.docx	Defines Sacred Site Anchors, LoopCache, and Personal Loop Charms.
LOOPtLOOP Addendum - Loop Bootstrap Seed - Presence-Based API Expansion.docx	Defines metadata updates, permissions block, signal/status, optional loop directory, consent stamps, and signature chaining.
LOOPtLOOP -- Two-Way TOTP Consent Loop Implementation Overview.docx	Defines generate-seed, pair, reset-loop, mutual consent, temporal alignment, minimal infrastructure, and recoverability.
Loop-Based Presence Platform - Partner Brief.docx	Frames platform access, SDKs, NFC tokens, ESP32 anchors, and partner verticals.
index.html / WitnessMark v1	Current working witness UI with external LOOPtLOOP API placeholder seam.
https://github.com/bobrs/abracadoo-code	Abracadoo PWA codebase and ontology reference.